BetVector — Cloud Migration Strategy Report

Version 1.0 · March 2026

Executive Summary

BetVector is a production-grade football betting intelligence system running locally on macOS with GitHub Actions for scheduling. This report evaluates every viable path to move BetVector to the cloud — enabling multi-user access, eliminating SQLite limitations, and establishing a reliable hosting foundation for the testing and growth phases.

Key finding: BetVector's architecture is already cloud-ready. The SQLAlchemy ORM, config-driven design, and `user_id`-scoped database schema mean the migration is primarily an infrastructure decision, not a code rewrite. The single most impactful change is moving from SQLite-in-git to PostgreSQL — this unlocks every hosting option and eliminates binary merge conflicts in GitHub Actions.

Recommended path:

1. **Phase 1 (Now, $0/mo):** Streamlit Community Cloud + Neon PostgreSQL + GitHub Actions
2. **Phase 2 (Growth, ~$10-25/mo):** Railway all-in-one PaaS
3. **Phase 3 (Scale, ~$40-70/mo):** Fly.io with global distribution

§1 — Current Architecture Analysis

What runs today

Component	Current Solution	Limitation
Database	SQLite 9.1 MB, WAL mode, committed to git	Binary merge conflicts, single-writer, no concurrent access
Dashboard	Streamlit 1.28+ on localhost:8501	Only accessible locally, password gate
Pipelines	3 GitHub Actions cron jobs (06:00, 13:00, 22:00 UTC)	Actions commits .db back to git, 540+ lines of migration hacks
Email	Gmail SMTP via smtplib	Works fine, no change needed
Config	YAML files + .env	Works fine, cloud-compatible
Auth	Single shared password (DASHBOARD_PASSWORD)	No per-user identity

Resource requirements (measured from codebase analysis)

Metric	Pipeline (morning)	Dashboard (10 users)	Both on one instance
RAM (minimum)	512 MB	512 MB	1.5 GB
RAM (recommended)	1 GB	1 GB	2 GB
CPU	1 vCPU (I/O bound)	1 vCPU (GIL-bound)	2 vCPU
Disk	3 GB minimum	3 GB minimum	10 GB recommended
Runtime	2-10 min per run	Always-on	Always-on + 3x burst
Network	Outbound HTTPS to 9 APIs	Inbound port 8501	Both

System dependencies for containerisation

The entire stack runs on `python:3.11-slim` with one additional system package:

```
apt-get install -y --no-install-recommends libgomp1
```

This single line covers xgboost and lightgbm. All other packages (pandas, scipy, statsmodels, streamlit, plotly) install cleanly from PyPI wheels with zero system dependencies.

---

§2 — PostgreSQL Migration (The Critical Prerequisite)

Moving from SQLite to PostgreSQL is the **single most impactful change** and prerequisite for every cloud architecture. The CLAUDE.md claim "only the connection string changes" is 95% true.

What actually needs to change

File	Change	Effort
`requirements.txt`	Add `psycopg2-binary==2.9.10`	1 line
`src/database/db.py` line ~148	Guard `mkdir` call to SQLite-only	3 lines
`src/database/models.py`	Replace 25× `server_default=sa_text("(datetime('now'))")` with `func.now()`	25 lines
`.github/workflows/*.yml`	Remove "Commit database changes" steps, remove inline `sqlite3` migrations	Delete ~540 lines
New: `alembic.ini` + `migrations/`	Replace ad-hoc PRAGMA migrations with Alembic	New file
`config/settings.yaml`	Add `DATABASE_URL` env var support	2 lines

What does NOT change

- All 23 ORM models — zero changes (standard SQL types, no SQLite-specific constructs)
- All scrapers, feature engineers, models, evaluation — zero changes
- All dashboard pages — zero changes
- Pipeline orchestrator — zero changes (reads/writes via SQLAlchemy session)

Effort estimate: 2-4 hours

The `datetime('now')` server defaults are the one real migration issue. PostgreSQL uses `now()` or `CURRENT_TIMESTAMP`. The clean fix is replacing all 25 instances with SQLAlchemy's dialect-agnostic `func.now()`.

Workflow simplification

After PostgreSQL, the morning.yml workflow drops from 288 lines to ~30:

```yaml
name: Morning Pipeline
on:
  schedule:
    - cron: "0 6 * * *"
jobs:
  morning:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    env:
      DATABASE_URL: ${{ secrets.DATABASE_URL }}
      GMAIL_APP_PASSWORD: ${{ secrets.GMAIL_APP_PASSWORD }}
      THE_ODDS_API_KEY: ${{ secrets.THE_ODDS_API_KEY }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
```

```
      - run: pip install -r requirements.txt && pip install -e .
      - run: python run_pipeline.py morning
```

No SQLite migrations. No binary file commits. No merge conflict handling.

---

§3 — Cloud Platform Evaluation

Managed PostgreSQL (Free Tiers)

Provider	Free Storage	Compute	Sleep Behaviour	Always Free?	Best Fo
Neon	512 MB	191.9 hrs/mo (0.25 vCPU)	Scales to zero after 5 min, <500ms cold start	Yes	BetVect (recomr
Supabase	500 MB	Shared nano	Pauses after 1 week inactivity	Yes	Active p (daily pi keep it a
CockroachDB	10 GiB	50M request units/mo	No sleep	Yes	If storag become concerr
Aiven	1 GB	Limited (25 connections)	No SLA	Yes	Backup
Tembo	10 GB	1 vCPU, 1 GB RAM	No sleep reported	Verify	Generou persists
ElephantSQL	—	—	—	**Shut down Jan 2025**	Do not u

Recommendation: Neon. Serverless architecture, sub-second cold starts, 512 MB storage (BetVector needs ~50 MB), point-in-time restore via branches, and standard PostgreSQL wire

protocol. SQLAlchemy works with `psycopg2` driver — set `pool_pre_ping=True` and reduce pool size for serverless connections.

App Hosting Platforms

Streamlit Community Cloud (Free)

Aspect	Detail
Cost	$0/mo
RAM	~1 GB shared
Sleep	After several days of no visitors (wake-up ~10 sec)
Secrets	Built-in secrets management
Deployment	Direct from GitHub repo (1-click)
PostgreSQL	None — bring your own (Neon)
Cron jobs	None — keep GitHub Actions
Private apps	1 private app per account

Verdict: Strong for Phase 1. Handles the dashboard layer for free. Sleep behaviour is acceptable during testing phase — daily visits wake it in seconds.

Railway (Hobby Plan ~$5-10/mo)

Aspect	Detail
Base cost	$5/mo flat + usage (~$0.000463/vCPU-minute)
Total for BetVector	~$10-18/mo (web + DB + 3 cron jobs)
RAM	Up to 8 GB per service
Sleep	None — always-on

Aspect	Detail
Secrets	First-class env var support
PostgreSQL	Managed plugin included
Cron jobs	Native support, define schedule per service
Deployment	Auto-deploy on GitHub push
DX Score	9/10

Verdict: Best all-in-one platform for Phase 2. Single vendor, excellent developer experience, native cron, managed PostgreSQL. Migration from Phase 1 takes ~10 minutes — push code, set DB URL, done.

Fly.io (Free Allowance + Usage)

Aspect	Detail
Free compute	3 shared-CPU VMs (256 MB each)
Free storage	3 GB persistent volume
Free bandwidth	160 GB/mo outbound
Total for BetVector	$10-17/mo (minimal) or $40-70/mo (growth)
Cron jobs	No built-in — use Fly Machines on schedule or external trigger
DX Score	6.5/10 (CLI-first, steeper learning curve)

Verdict: Best for Phase 3 when global distribution and auto-scaling matter. 256 MB RAM per VM may be tight for Python's scientific stack — monitor carefully.

Render

Aspect	Detail
Free web service	Spins down after 15 min inactivity (30-60 sec cold start)
Free PostgreSQL	**Expires after 90 days** (deleted unless upgraded)
Paid total	$34/mo minimum for reliable operation

Verdict: Not recommended. The 90-day PostgreSQL expiry is a showstopper. Free tier spin-down makes Streamlit unusable.

Koyeb (Free Nano)

Aspect	Detail
Free tier	1 nano instance: 0.1 vCPU, 512 MB RAM, always-on
Sleep	None on free tier (always-on)
PostgreSQL	None — bring your own

Verdict: Viable alternative to Streamlit Community Cloud for hosting the dashboard. 512 MB RAM is tight but workable for Streamlit alone. No cron support on free tier.

VPS Options

Provider	Cheapest Tier	RAM	CPU	Storage	Monthly Cost	Always Free
Oracle Cloud	ARM A1	24 GB	4 OCPU	200 GB	$0	Yes (but hard to provision)
GCP	e2-micro	1 GB	2 vCPU (burst)	30 GB HDD	$0	Yes (US regions only)
Hetzner	CAX11 ARM	4 GB	2 vCPU	40 GB NVMe	~$4/mo	No

Provider	Cheapest Tier	RAM	CPU	Storage	Monthly Cost	Always Free
DigitalOcean	Basic Droplet	1 GB	1 vCPU	25 GB SSD	$6/mo	No
AWS	t3.micro	1 GB	2 vCPU	30 GB	$0 (12 mo only)	No — expire
Vultr/Linode	Smallest	1 GB	1 vCPU	25 GB	$5-6/mo	No

Oracle Cloud Always Free: Most powerful free option (4 OCPU, 24 GB RAM, 200 GB disk) but availability is severely constrained — provisioning ARM instances may take days of repeated attempts. Developer experience is poor (DX: 1/10). High lock-in risk.

Hetzner CAX11: Best cost-to-capability ratio of any paid option. 4 GB RAM for ~$4/mo runs everything comfortably: Streamlit + PostgreSQL + cron jobs. Full root access, no artificial constraints. Requires Linux sysadmin skills.

GCP e2-micro: 1 GB RAM is genuinely tight for Python's scientific stack. Suitable for pipeline execution only, not for hosting both dashboard and pipelines. The 3 free Cloud Scheduler jobs coincidentally match BetVector's 3 daily pipelines.

§4 — Recommended Architectures (Ranked)

Architecture A: Hybrid Free Tier ($0/mo) — RECOMMENDED FOR NOW

```
┌──────────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│  Streamlit Community │──►│  Neon PostgreSQL  │◄──│  GitHub Actions  │
│       Cloud          │     │   (free tier)     │     │  (3 cron jobs)   │
│   (dashboard)        │     │   512 MB storage  │     │  morning/mid/eve │
└──────────────────────┘     └──────────────────┘     └──────────────────┘
```

Component	Service	Cost
Dashboard	Streamlit Community Cloud	$0
Database	Neon free PostgreSQL	$0
Pipelines	GitHub Actions (existing)	$0
Email	Gmail SMTP (existing)	$0
Secrets	GitHub Secrets + Streamlit Secrets	$0
Total		**$0/mo**

Pros:

- Zero cost, zero financial risk
- Leverages existing GitHub Actions workflows (already working)
- Neon's serverless Postgres eliminates SQLite's single-writer limitation
- Multi-user access immediately — dashboard is publicly accessible
- No infrastructure to maintain
- Pipeline results available to dashboard within seconds (shared DB)

Cons:

- Streamlit Community Cloud sleeps after inactivity (10 sec wake-up)
- GitHub Actions cron can have 15-30 min delays during peak load
- Free tier policies can change (monitor quarterly)
- 1 private app limit on Streamlit Community Cloud (use password gate for auth)

Migration effort: ~4 hours (PostgreSQL migration + deploy to Streamlit Cloud)

Architecture B: Railway All-in-One (~$10-25/mo) — RECOMMENDED FOR GROWTH

```
┌──────────────────────────────────────────────┐
│                  Railway                     │
│  ┌────────────┐  ┌──────────┐  ┌──────────┐ │
│  │ Streamlit  │  │ PostgreSQL│  │ 3× Cron  │ │
│  │ Web Svc    │  │  Plugin  │  │ Services │ │
│  │(always-on) │  │ (managed)│  │(scheduled)│ │
│  └────────────┘  └──────────┘  └──────────┘ │
└──────────────────────────────────────────────┘
```

Component	Service	Cost
Dashboard	Railway Web Service (256 MB RAM)	~$5-8/mo
Database	Railway PostgreSQL Plugin (1 GB)	~$5-7/mo
Pipelines	Railway Cron Services (3×)	~$2-3/mo
Email	Gmail SMTP (unchanged)	$0
Secrets	Railway Environment Variables	Included
Total		**$12-18/mo**

Pros:

- Always-on, no sleep or cold starts
- Single vendor — one dashboard for everything
- Native cron support — no more GitHub Actions workflow complexity
- Auto-deploy on git push
- Managed PostgreSQL with daily backups
- Best developer experience (DX: 9/10)

Cons:

- $12-18/mo recurring cost
- Railway-proprietary deployment format (medium lock-in)

- 99.5% uptime (not SLA-guaranteed)

Migration from Architecture A: ~10 minutes (push to Railway, point DATABASE_URL to Railway Postgres or keep Neon)

Architecture C: Fly.io Global (~$15-40/mo) — FOR SCALE

Component	Service	Cost
Dashboard	Fly Machine (shared-cpu-2x, 512 MB)	~$3-10/mo
Database	Neon PostgreSQL (or Fly Postgres)	$0-12/mo
Pipelines	Fly Scheduled Machines (3×)	~$2-5/mo
Total		**$10-17/mo (min), $40-70/mo (growth)**

Pros:

- Multi-region deployment (global edge)
- Excellent auto-scaling
- No cold starts (Fly Machines are warm)
- 99.9% uptime (best-effort)

Cons:

- CLI-first workflow (steeper learning curve)
- No built-in cron — requires Fly Machines scheduling or external trigger
- Egress bandwidth charges ($0.02/GB) can add up
- Fly Postgres is self-managed (reliability complaints)

Architecture D: Hetzner VPS (~$4/mo) — FOR FULL CONTROL

Component	Service	Cost
Everything	Hetzner CAX11 (4 GB RAM, 2 ARM vCPU)	~$4/mo

Component	Service	Cost
Dashboard	Streamlit via systemd + Caddy reverse proxy	Included
Database	PostgreSQL 16 self-hosted	Included
Pipelines	systemd timers (3×)	Included
Total		**~$4/mo**

Pros:

- Best cost-to-capability ratio ($4 for 4 GB RAM, 2 vCPU, 40 GB SSD)
- Full root access, no artificial constraints
- No vendor lock-in (standard Linux)
- No free tier policy risk
- Fastest possible deployment (all services local)

Cons:

- You manage OS updates, security patches, SSL certs, backups
- Manual deployment pipeline (or set up Coolify for UI)
- No HA or failover without extra work
- Requires Linux sysadmin knowledge (DX: 2/10)

---

§5 — Comparative Pricing Matrix

Monthly Cost at Different Scales

Architecture	1-5 Users	10-50 Users	100+ Users	DX Score
A: Hybrid Free	$0	$0-5	Not suitable	9/10

Architecture	1-5 Users	10-50 Users	100+ Users	DX Score
B: Railway	$12-18	$21-35	$50-80	9/10
C: Fly.io	$10-17	$40-70	$80-150	6.5/10
D: Hetzner VPS	$4	$4-12	$12-30	2/10
AWS (ECS+RDS)	$22-30	$70-110	$150+	4/10
GCP (Run+SQL)	$25-40	$81-122	$150+	5/10
Azure	$33-52	$82-135	$200+	3/10

18-Month Cost Projection

Scenario	Strategy	Total Cost
A: Stay Free	Hybrid forever	$0
B: Free → Railway at month 6	Hybrid → Railway	$300
C: Free → Railway → Fly.io	Hybrid → Railway (6 mo) → Fly.io (6 mo)	$450
D: Jump to AWS	AWS from day 1	$780+

---

§6 — Platform Reliability Comparison

Platform	Uptime SLA	Cold Starts	Sleep Policy	Pipeline Reliability
Streamlit Cloud	~99.5% (community)	10 sec wake-up	After days of no visits	N/A (no cron)
GitHub Actions	99.95%	30-60 sec runner start	N/A	15-30 min delays at peak

Platform	Uptime SLA	Cold Starts	Sleep Policy	Pipeline Reliability
Neon	99.95%	<500 ms	Scales to zero after 5 min	Always available
Railway	99.5% (not guaranteed)	None (always-on)	None	Native cron, reliable
Fly.io	99.9% (best-effort)	None (warm machines)	None	Scheduled machines
Hetzner	99.5% (not guaranteed)	None (bare metal)	None	systemd timers, most reliable
Render	99.95% (paid SLA)	30-60 sec (free)	After 15 min inactivity	Cold start delays
Oracle Cloud	99.5%	None	Auto-stop after 7 days idle	Unreliable free tier

---

§7 — Multi-User Authentication Strategy

Current state

The dashboard has a single shared password (`DASHBOARD_PASSWORD`) with no per-user identity. Auth is stored in `st.session_state["authenticated"]`. The database schema already has `user_id` on all personal tables (bet_log, bankroll, notifications).

Recommended upgrade path

Phase	Approach	Users	Code Changes	Cost
Testing (now)	Shared password (current)	2-5 trusted	None	$0

Phase	Approach	Users	Code Changes	Cost
Growth	`streamlit-authenticator`	5-20	~50 lines + users.yaml	$0
Production	Supabase Auth or Cloudflare Access	Unlimited	~100 lines	$0 (free tier)

What the codebase needs for true multi-user

1. Replace `user_id=1` hardcode in dashboard.py + page modules with session-based user resolution
2. Add password column (hashed) to User model
3. Thread `user_id` through all page queries that currently hardcode `1`
4. Add `WHERE user_id = :uid` filters to bet_log, bankroll, value_bets queries

The schema is already prepared — this is a query filter change, not a schema change.

---

§8 — Migration Roadmap

Phase 1: PostgreSQL Migration + Free Tier Deployment (Week 1)

Effort: ~4 hours

1. Add `psycopg2-binary` to requirements.txt
2. Replace 25 `datetime('now')` server defaults with `func.now()` in models.py
3. Guard SQLite-specific `mkdir` in db.py
4. Add `DATABASE_URL` environment variable support to db.py
5. Provision Neon free PostgreSQL instance
6. Run schema creation against Neon (`python run_pipeline.py setup`)
7. Migrate existing data: SQLite → PostgreSQL (pgloader or custom script)
8. Deploy dashboard to Streamlit Community Cloud

9. Update GitHub Actions workflows to connect to Neon
10. Remove `git add data/betvector.db` from all workflows

Validation:

- Morning pipeline runs via GitHub Actions, writes to Neon PostgreSQL
- Dashboard on Streamlit Community Cloud reads from same Neon instance
- Share dashboard URL with 2-3 test users
- Verify password gate works on public deployment

Phase 2: Railway Migration (Month 3-6)

Trigger: When free tier limitations become frustrating

- Streamlit sleep disrupts user access
- GitHub Actions delays affect morning email timing
- You want a single-vendor operational view

Effort: ~1 hour

1. Create Railway project, connect GitHub repo
2. Add PostgreSQL plugin (or keep Neon)
3. Set environment variables
4. Configure 3 cron services (morning/midday/evening)
5. DNS switch (if using custom domain)

Phase 3: Production Scale (Month 6+)

Trigger: When you have 10+ regular users or need global access

Options:

- **Stay on Railway** — just increase resources ($25-35/mo)
- **Move to Fly.io** — for global distribution ($40-70/mo)
- **Move to Hetzner VPS** — for maximum control ($4-12/mo)

§9 — Platform-Specific Gotchas

Neon PostgreSQL

- Use `pool_pre_ping=True` in SQLAlchemy engine config (handles connection drops on scale-to-zero)
- Reduce pool size to 3-5 connections (free tier limit)
- The `psycopg2` driver works fine; no special serverless driver needed for this workload
- Neon branches give you free staging environments — branch before schema changes

Streamlit Community Cloud

- App must be in a GitHub repo (private requires Pro account; use password gate for access control)
- No file system persistence between restarts — this is fine since everything is in PostgreSQL
- Secrets are configured via the web UI (Settings → Secrets), stored as TOML format
- Maximum ~1 GB RAM — sufficient for dashboard but not for pipeline execution

Railway

- Hobby plan requires credit card even with $5 free credit
- Each cron service is a separate container — environment variables must be duplicated or linked
- PostgreSQL plugin provisions a shared instance — not suitable for production SLA, but fine for testing
- Monitor compute usage: 3 cron jobs × 10 min × 3x/day = ~90 minutes/day of compute

GitHub Actions (current, retained in Phase 1)

- Cron schedules can be delayed 15-30 minutes during peak GitHub load (mornings EST)
- Actions on free plan get 2,000 minutes/month; your 3 daily pipelines use ~900 minutes/month (plenty of headroom)

- Python dependency caching (`setup-python` with `cache: pip`) reduces install time from ~90 sec to ~10 sec

---

§10 — Security Considerations

Secrets management across platforms

Secret	Current Location	Phase 1 Location	Phase 2 Location
`DATABASE_URL`	N/A (local SQLite)	GitHub Secrets + Streamlit Secrets	Railway env vars
`GMAIL_APP_PASSWORD`	`.env` file	GitHub Secrets	Railway env vars
`DASHBOARD_PASSWORD`	`.env` file	Streamlit Secrets	Railway env vars
`THE_ODDS_API_KEY`	`.env` file	GitHub Secrets	Railway env vars
`API_FOOTBALL_KEY`	`.env` file	GitHub Secrets	Railway env vars

Key security principles for cloud deployment

- Never commit `.env` or secrets to git (already enforced via .gitignore)
- Use HTTPS for all dashboard access (Streamlit Cloud provides this; Railway provides this; Neon connections use TLS by default)
- Rotate database credentials quarterly
- Set Neon IP allowlist if using static GitHub Actions runners
- Keep the password gate on the dashboard even when publicly accessible
- Consider Cloudflare Access (free for up to 50 users) as a zero-trust gateway in front of the dashboard

---

§11 — Decision Framework

When to upgrade from Phase 1 to Phase 2

Upgrade to Railway when ANY of these become true:

```
IF dashboard_sleep_disrupts_users
  OR github_actions_delays_affect_morning_emails
  OR you_want_single_vendor_simplicity
  OR you_need_24/7_always_on_dashboard
THEN upgrade_to_railway ($12-18/mo)
```

When to upgrade from Phase 2 to Phase 3

Upgrade to Fly.io or scale Railway when:

```
IF users_span_multiple_continents
  OR you_need_99.9%_uptime
  OR you_need_horizontal_autoscaling
  OR you_hit_railway_resource_limits
THEN upgrade_to_flyio_or_scale_railway ($40-70/mo)
```

Platforms to avoid

Platform	Reason
Oracle Cloud Always Free	Extremely difficult to provision ARM instances, Oracle-proprietary tooling (DX: 1/10), auto-stop after 7 days idle, extreme lock-in
Render (free tier)	90-day PostgreSQL expiry is a showstopper, 15-min spin-down kills Streamlit usability

Platform	Reason
AWS/GCP/Azure (native)	Massive over-engineering for this scale ($25-135/mo), steep learning curves, high lock-in
Heroku	No free tier since Nov 2022, minimum $12/mo for basic functionality

---

§12 — Final Recommendation

Start with Architecture A (Hybrid Free Tier)

The migration effort is approximately 4 hours:

1. Add `psycopg2-binary` to requirements
2. Fix 25 `datetime('now')` server defaults in models.py
3. Add `DATABASE_URL` env var support to db.py
4. Provision Neon free PostgreSQL
5. Migrate 9.1 MB of data
6. Deploy dashboard to Streamlit Community Cloud
7. Simplify GitHub Actions workflows
8. Remove `data/betvector.db` from git tracking

Then, when testing with multiple users reveals the limitations of the free tier, upgrade to Railway (~$10-25/mo). The transition from A to B requires zero code changes — just moving the Streamlit process from Community Cloud to Railway and pointing the same `DATABASE_URL` at Railway's managed PostgreSQL (or keep Neon).

The single most impactful change is SQLite → PostgreSQL. It eliminates binary merge conflicts, enables true multi-user concurrent access, removes 540+ lines of SQLite migration hacks from GitHub Actions, and unlocks every cloud hosting option.

Appendix A — Docker Configuration

```dockerfile
FROM python:3.11-slim

# Only system dependency needed for xgboost + lightgbm
RUN apt-get update && \
    apt-get install -y --no-install-recommends libgomp1 && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Install Python dependencies first (layer caching)
COPY requirements.txt pyproject.toml ./
RUN pip install --no-cache-dir -r requirements.txt && pip install -e .

# Copy application code
COPY . .

# Streamlit configuration
ENV MPLBACKEND=Agg
EXPOSE 8501

CMD ["streamlit", "run", "src/delivery/dashboard.py", \
     "--server.port=8501", "--server.address=0.0.0.0", \
     "--server.headless=true"]
```

Appendix B — Neon Connection Configuration

```python
# src/database/db.py — PostgreSQL-compatible engine creation
from sqlalchemy import create_engine

engine = create_engine(
```

```
    os.environ["DATABASE_URL"],
    pool_pre_ping=True,      # Handle serverless connection drops
    pool_size=3,              # Conservative for Neon free tier
    max_overflow=2,           # Allow brief burst connections
    pool_recycle=300,         # Recycle connections every 5 min
)
```

Appendix C — Simplified GitHub Actions Workflow (Post-Migration)

```yaml
name: Morning Pipeline
on:
  schedule:
    - cron: "0 6 * * *"
  workflow_dispatch:

concurrency:
  group: betvector-morning
  cancel-in-progress: false

jobs:
  morning:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    env:
      DATABASE_URL: ${{ secrets.DATABASE_URL }}
      GMAIL_APP_PASSWORD: ${{ secrets.GMAIL_APP_PASSWORD }}
      FROM_EMAIL: ${{ secrets.FROM_EMAIL }}
      API_FOOTBALL_KEY: ${{ secrets.API_FOOTBALL_KEY }}
      FOOTBALL_DATA_ORG_KEY: ${{ secrets.FOOTBALL_DATA_ORG_KEY }}
      THE_ODDS_API_KEY: ${{ secrets.THE_ODDS_API_KEY }}
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
      - name: Install dependencies
        run: pip install -r requirements.txt && pip install -e .
      - name: Run morning pipeline
        run: python run_pipeline.py morning
```

Appendix D — Railway Configuration

```toml
# railway.toml
[build]
builder = "DOCKERFILE"
dockerfilePath = "Dockerfile"

[deploy]
startCommand = "streamlit run src/delivery/dashboard.py --server.port=$PORT --server
healthcheckPath = "/"
healthcheckTimeout = 30
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 3
```

Appendix E — Verification Checklist

Before making a final platform decision, verify these items directly on each platform's pricing page (free tier policies change frequently):

- ☐ Neon free tier: Confirm 512 MB storage and 191.9 compute-hours/mo at neon.tech/pricing

- ☐ Supabase free tier: Confirm 500 MB storage and pause behaviour at supabase.com/pricing
- ☐ Streamlit Community Cloud: Confirm free tier for private apps at streamlit.io/cloud
- ☐ Railway Hobby plan: Confirm $5/mo base and cron availability at railway.app/pricing
- ☐ GitHub Actions: Confirm 2,000 min/mo free at github.com/pricing
- ☐ Fly.io free allowance: Confirm 3 shared-CPU VMs at fly.io/docs/about/pricing